

Smart Flight Filter API

A traveller types what they want in plain language. This service returns the exact filter JSON the flight listing page already consumes — complete, every key present, ready to apply unchanged.

BASE URL <code>http://aimltest1.easemytrip.com</code>	INTERACTIVE DOCS /docs	AUTH None
STATE Stateless		

01 What it does

The listing page runs a normal flight search. The traveller then types something like “*give me flights under 10k, non-stop, IndiGo*”. You post that raw sentence here and get back the filter object your existing filter call already accepts.

The scope is deliberately narrow: a sentence in, the filter JSON out. This service does not run the search, does not count results, does not render chips, and never predicts whether a filter will return flights. Everything downstream stays yours.

THE ONE THING WE NEED FROM YOU

Values like airline names, layover airports and aircraft types only exist *after* a search runs — which is exactly why you render those chips post-search. So **you send us the same option lists you already computed**, and we only ever emit a value that came from that list. That is what stops the model inventing an airline that does not fly the route.

02 Endpoints

POST /API/SMART-FILTER/PARSE

Content-Type: application/json

The only endpoint you call. Takes the chat message plus context, returns the filter JSON.

GET /HEALTH

```
{ "status": "ok" }
```

Liveness probe. Use it for monitoring.

GET /DOCS

Swagger UI. Try requests straight from the browser.

There is no authentication header.

Browser calls are restricted to one origin: `https://www.easemytrip.com` . Any other origin — a staging host, `m.easemytrip.com` , or the bare domain without `www` — is refused by the browser. Tell us the origins you actually serve from and we will add them. Calling from your own backend is unaffected; CORS applies only to browsers.

03 Request

FIELD	REQUIRED	NOTES
message	YES	The raw user text, unmodified . Do not pre-clean or pre-extract it — route, dates, passenger counts and pleasantries are recognised and ignored on our side.
tripType	YES	"oneway" or "roundtrip" . Decides whether <code>filter</code> comes back as one object or a two-element array.
facets	YES	Your option lists. Missing or empty → <code>400</code> . See §04.
currentFilter	OPTIONAL	The <code>filter</code> from the previous response. Send it back so refinements stack (<i>"also make it morning"</i>). Send <code>null</code> for a fresh filter.
provider	OPTIONAL	Override the model provider for one call. Leave it out.
model	OPTIONAL	Override the model name for one call. Leave it out.

STATELESS – YOU OWN THE CONVERSATION

We keep nothing between calls. Cumulative refinement works *only* because you round-trip `currentFilter` . If you send `null` every time, each message starts from a blank filter and the previous one is lost.

04 Facets — the five lists you send

Five fields, each a **comma-separated string**. These are the only values we are permitted to write into the filter; whatever token you put here is echoed back **verbatim**.

FACETS

```
{
  "airline":      "IndiGo,Air India,AkasaAir,Air India Express,SpiceJet",
  "layover":      "HYD:Hyderabad,BLR:Bengaluru,AMD:Ahmedabad",
  "airCarftType": "Boeing 737 (Narrow-body),Airbus A320 (Narrow-body)",
  "takeOffAirport": "DEL:New Delhi,DXN:Noida,HDO:Ghaziabad",
  "landingAirport": "BOM:Mumbai,NMI:Navi Mumbai"
}
```

The `CODE:Label` form

When the value your filter API wants differs from the word a traveller would say, write both, separated by a colon. `"BOM:Mumbai"` means: **match** the user's word against *Mumbai*, but **emit** `"BOM"`. Where the value and the label are the same word, `"IndiGo"` on its own is fine.

This is how the code-versus-name question disappears. If your filter API expects `"6E"` rather than `"IndiGo"`, send `"6E:IndiGo"` and `"6E"` is what comes back.

Arrays also accepted

If you already hold structured lists, you may send an array of strings or of `{"value","label"}` objects instead of a comma-separated string. Same result.

NOT FACETS

Stops, price, duration and times need no facet. A stop count is just an integer and a price is just a number, so we can always emit those. Send only the five lists above — nothing else is read.

What happens when a value is not in your list

If the traveller asks for an airline you did not send, we have no token to write, so we do not guess. Depending on whether anything *else* in the message landed, you get `no_match` or a partial-apply — see §05.

05 Response

200 OK

```
{
  "status": "applied",      // "applied" | "no_match"
  "filter": { ... },       // object (oneway) or 2-element array (roundtrip)
  "message": ""            // text to show the traveller, or ""
}
```

Three fields, always. `filter` is always **complete** — every key present, unused ones at their defaults — so you can drop it into your existing filter call without merging anything.

Branching on `status`

STATUS	WHAT IT MEANS	WHAT YOU DO
applied	At least one preference landed. <code>filter</code> is the new filter.	Apply <code>filter</code> . If <code>message</code> is non-empty, show it too.
no_match	Nothing landed. <code>filter</code> comes back as the <code>currentFilter</code> you sent, untouched .	Show <code>message</code> . Leave the traveller's existing filters alone.

WHY NO_MATCH RETURNS YOUR FILTER UNCHANGED

If it returned a blank filter instead, every “thanks!” would silently wipe the filters the traveller had already applied. You can safely apply the returned `filter` in both cases — it is never destructive.

The three messages

MESSAGE TEXT	STATUS	TRIGGERED BY
No filter change requested.	no_match	The message carried no filter intent — a search (“ <i>delhi to mumbai tomorrow</i> ”), a greeting, chit-chat, or an empty string.
No flights match this preference.	no_match	A real preference, but every value asked for was outside your facets (“ <i>only Vistara</i> ” when Vistara is not in the list).
We applied the closest match so you still see flights.	applied	Partial — some preferences landed, at least one did not (“ <i>under 10k on Vistara</i> ”: the price applies, the airline does not).

The wording lives in one place on our side. Ask and we will change it; you can also ignore `message` entirely and write your own copy off `status`.

06 The filter JSON

25 keys, always all present. Every `Is*` flag is computed by us from its payload, so a populated `Stop` can never arrive with `IsStop: false`.

```
{
  "IsPrice": false,          "Price": { "Min": "", "Max": "" },
  "IsDepTime": false,       "DepTime": { "Min": "", "Max": "" },
  "IsArrTime": false,       "ArrTime": { "Min": "", "Max": "" },
  "IsDuration": false,      "Duration": { "Min": "", "Max": "" },
  "IsStop": false,          "Stop": [],
  "IsAirline": false,       "Airline": [],
  "IsLayover": false,       "Layover": [],
  "IsAirCarftType": false,  "AirCarftType": [],
  "IsTakeOffAirport": false, "TakeOffAirport": [],
  "IsLandingAirport": false, "LandingAirport": [],
  "IsSortBy": false,        "SortBy": "",
  "Refundable": false,
  "IsWifi": false,
  "IsRedEyes": false
}
```

SPELLING IS THE WIRE FORMAT

`AirCarftType` and `IsRedEyes` are spelled exactly as your existing API spells them. They are intentionally not “corrected”. If your filter API is ever fixed, tell us and we change it in one place.

`Refundable`, `IsWifi` and `IsRedEyes` are standalone booleans with no companion array and no `Is` twin of their own — note that `Refundable` alone carries no `Is` prefix.

07 Value formats

This is the table to check against your filter API before we go live.

FIELD	EMITTED AS	EXAMPLE
Price	Rupees, as a string	<code>{"Min":"","Max":"10000"}</code>
DepTime ArrTime	<code>HH:MM</code> , 24-hour	<code>{"Min":"06:00","Max":"12:00"}</code>
Duration	Minutes , as a string	<code>{"Min":"","Max":"120"}</code>
Stop	Array of integers . Capped at 3.	<code>[0, 1]</code>
Airline Layover AirCarftType TakeOffAirport LandingAirport	Array of strings, echoed verbatim from the facet value you sent	<code>["IndiGo"]</code> <code>["BOM"]</code>
Refundable IsWifi IsRedEyes	Boolean	<code>true</code>
SortBy	One label string — see §08	<code>"Cheapest"</code>

Night windows wrap past midnight

“*night flights*” emits `DepTime { "Min": "21:00", "Max": "03:00" }` — `Min` greater than `Max`, deliberately. Confirm your filter treats that as a wrap rather than an empty window. If it does not, tell us and we will emit it differently.

Numbers are passed through, not clamped

“*under 2k*” on a route starting at ₹4,210 emits `Price.Max = "2000"` exactly as asked. We do not clamp it upward and we do not guess that it will return nothing — you run the filter and handle the empty state.

08 SortBy

Comparative words set the sort rather than inventing a price band, so “*cheapest*” becomes `SortBy: "Cheapest"` — not a guessed `Price.Max`. The nine values match your existing sort chips exactly:

SORTBY — ALLOWED VALUES

"Best"	"Cheapest"	"Fastest"
"Early Take-off"	"Late Take-off"	"Early Arrival"
"Late Arrival"	"Slowest"	"Highest Price"

`IsSortBy` is `true` whenever `SortBy` is non-empty.

On a roundtrip there is one sort for the whole itinerary. Both legs always carry the same `SortBy` value, even when the traveller names a leg (“*sort the outbound by cheapest*”) — the results are sorted as one combined set, so the two legs can never disagree.

09 Round trip

With `tripType: "roundtrip"`, both `facets` and `currentFilter` become **two-element arrays** — `[outbound, return]` — and `filter` comes back the same way.

PER-LEG FACETS ARE NOT OPTIONAL

The two legs genuinely differ. On a live DEL↔BOM search the outbound offered layovers the return did not, and the price ranges were ₹6,270–44,266 against ₹5,980–26,195. One shared facet list would mis-filter a leg.

Leg targeting

A preference applies to **both legs** unless the traveller says otherwise. Words like *onward*, *outbound* and *departing* narrow it to index 0; *return*, *inbound* and *coming back* narrow it to index 1. So “*return should land before noon*” changes element 1 and leaves element 0 exactly as it was.

10 Worked examples

A. First filter — one way

POST /API/SMART-FILTER/PARSE

```
{
  "message": "show me flights under 10k, non stop, indigo",
  "tripType": "oneway",
  "currentFilter": null,
  "facets": {
    "airline": "IndiGo,Air India,AkasaAir,SpiceJet",
    "layover": "HYD:Hyderabad,BLR:Bengaluru",
    "airCarftType": "Boeing 737 (Narrow-body),Airbus A320 (Narrow-body)",
    "takeOffAirport": "DEL:New Delhi,DXN:Noida",
    "landingAirport": "BOM:Mumbai,NMI:Navi Mumbai"
  }
}
```

200 — STATUS: APPLIED

```
{
  "status": "applied",
  "message": "",
  "filter": {
    "IsPrice": true,    "Price": { "Min": "", "Max": "10000" },
    "IsStop": true,    "Stop": [0],
    "IsAirline": true, "Airline": ["IndiGo"],
    ... all other keys present at their defaults ...
  }
}
```

B. Refinement — send the filter back

The traveller then types “make it morning as well”. Post it with the previous `filter` as `currentFilter`:

POST /API/SMART-FILTER/PARSE

```
{
  "message": "make it morning as well",
  "tripType": "oneway",
  "currentFilter": { the filter from response A, verbatim },
  "facets": { same five lists }
}
```

```
{
  "status": "applied",
  "message": "",
  "filter": {
    "IsPrice": true,    "Price": { "Min": "", "Max": "10000" },
    "IsStop": true,    "Stop": [0],
    "IsAirline": true, "Airline": ["IndiGo"],
    "IsDepTime": true, "DepTime": { "Min": "06:00", "Max": "12:00" },
    ...
  }
}
```

“remove the airline filter” and “clear everything” work the same way — they clear rather than add.

C. Nothing to apply

MESSAGE: "DELHI TO MUMBAI TOMORROW"

```
{
  "status": "no_match",
  "message": "No filter change requested.",
  "filter": { exactly the currentFilter you sent }
}
```

D. Partial

MESSAGE: "UNDER 10K ON VISTARA" — VISTARA NOT IN FACETS

```
{
  "status": "applied",
  "message": "We applied the closest match so you still see flights.",
  "filter": {
    "IsPrice": true, "Price": { "Min": "", "Max": "10000" },
    Airline left untouched - we had no token to write
    ...
  }
}
```

11 Errors

CODE	CAUSE	FIX
400	<code>facets</code> missing, empty, or containing none of the five keys. For roundtrip, not a two-element array.	Send the facets. Never call this endpoint before a search has run.
422	Request body failed validation — a wrong type, or an unexpected extra field (the body is strict).	Match the field names in §03 exactly.
500	Server-side configuration fault (missing model credential).	Ours, not yours — report it and we fix it.

Anything else — a model timeout, a refusal, a garbled message — degrades to a `200` with `status: "no_match"` and the traveller's filters intact. **A failure on our side never breaks the page and never wipes their filters.**

12 Integration checklist

- 01 Open `/docs` and fire a request with your own facets. Confirm the shape before writing any code.
 - 02 Wire the five facet lists from whatever already builds your filter chips. Comma-separated strings; use `CODE:Label` wherever the emitted value differs from the displayed word.
 - 03 Take the returned `filter` and apply it to your existing filter call **unchanged**. Do not merge, reshape, or drop keys.
 - 04 Store the returned `filter` and send it back as `currentFilter` on the next message, so refinements stack.
 - 05 Branch on `status`. Show `message` whenever it is non-empty.
 - 06 **Confirm the value formats in §07 against your filter API** — particularly Duration in minutes, Price as a string, Stop as integers, and the midnight-wrapping night window.
 - 07 Tell us the exact tokens your filter API expects for airline and aircraft type, and put those in the facet `value` position. Then they come back verbatim and there is nothing left to map.
-

Smart Flight Filter API — integration specification, v1. Questions, format corrections and wording changes: Mayank Wadhwa.